Building APIs & MCP Servers

What is an API anyways?

Technically, API stands for Application Programming Interface. At some point or another, most large companies have built APIs for their customers, or for internal use.

An API isn't the same as the remote server—rather it is the part of the server that receives requests and sends responses.

When a company offers an API to their customers, it just means that they've built a set of dedicated URLs that return pure data responses—meaning the responses won't contain the kind of presentational overhead that you would expect in a graphical user interface like a website.

REST API

A REST API defines a set of functions which developers can perform requests and receive responses via HTTP protocol such as GET and POST.

REST stands for Representational state transfer which essentially refers to a style of web architecture that has many underlying characteristics and governs the behavior of clients and servers.

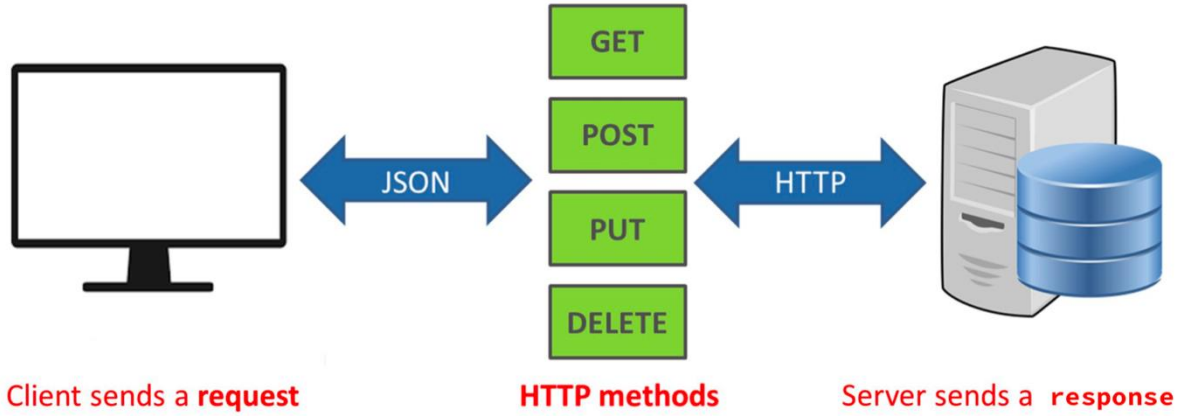
An API can be considered “RESTful” if it has the following features (main features of R):

Client–server – The client handles the front end the server handles the backend and can both be replaced independently of each other.

Stateless – No client data is stored on the server between requests and session state is stored on the client.

Cacheable – Clients can cache response (just like browsers caching static elements of a web page) to improve performance

REST API



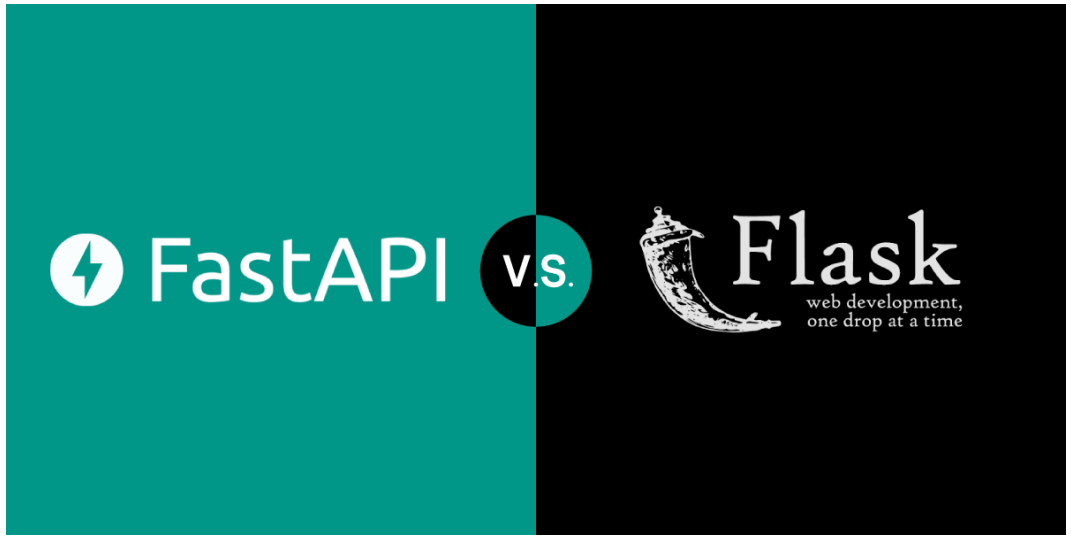

```
GET library.com/book
```

Why APIs Matter

Your model is useless if no one can use it.

APIs make your work accessible:
Streamlit apps can call your API
Mobile apps integrate models
Other services consume data
AI agents use tools via MCP

Essential for:
Separating concerns
Multiple clients
Versioning models
Scaling independently
Professional deployment



FastAPI uses a web server called ASGI or Asynchronous Server Gateway Interface, which is lighting fast because it's—well, you guessed it—Asynchronous. So, if you have a bunch of requests coming in, they don't have to wait for the other ones to complete before they are processed.

- Speed, as in development time and performance
- Decrease the number of bugs and errors in your code
- Build APIs from scratch

Flask uses a web server called WSGI, which stands for Web Server Gateway Interface and has been the Python standard for many years. The drawback is that it's synchronous. This means that if you have a bunch of requests, they have to wait in line for the queue to complete.

- A battle-tested framework, as it's been around for a long time
- Can develop quick prototypes
- Web application development

FastAPI Fundamentals

FastAPI

Modern, fast web framework for APIs

Key features:
Automatic documentation
Type hint validation
Async support
JSON serialization
Dependency injection
Built on Starlette/Pydantic

Why for data science:
Type safety
Auto docs
High performance
Input validation
Production ready

What are we doing with FastAPI?

We will create routes for our api that receives different requests (GET and POST), specifically a POSTing json data

Once our flask server receives json data it will apply a function and then return some value also in json format

FastAPI Code Structure

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello"}

@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

Run: `uvicorn main:app --reload`

Docs: /docs and /redoc

Loading Models

```
import joblib

app = FastAPI()
model = None

@app.on_event("startup")
def load_model():
    global model
    model = joblib.load("model.pkl")

@app.post("/predict")
def predict(features: list[float]):
    pred = model.predict([features])
    return {"prediction": float(pred[0])}
```

Load once at startup
Keep in memory
Fast predictions

Prediction Endpoints

```
class PredictionRequest(BaseModel):
    features: list[float]

class PredictionResponse(BaseModel):
    prediction: float
    confidence: float
    model_version: str

@app.post("/predict")
def predict(req: PredictionRequest):
    pred = model.predict([req.features])[0]
    conf = model.predict_proba([req.features]).max()
    return PredictionResponse(
        prediction=float(pred),
        confidence=float(conf),
        model_version="v1.0"
    )
```

Clear schemas
Include confidence
Version tracking

Automatic Documentation

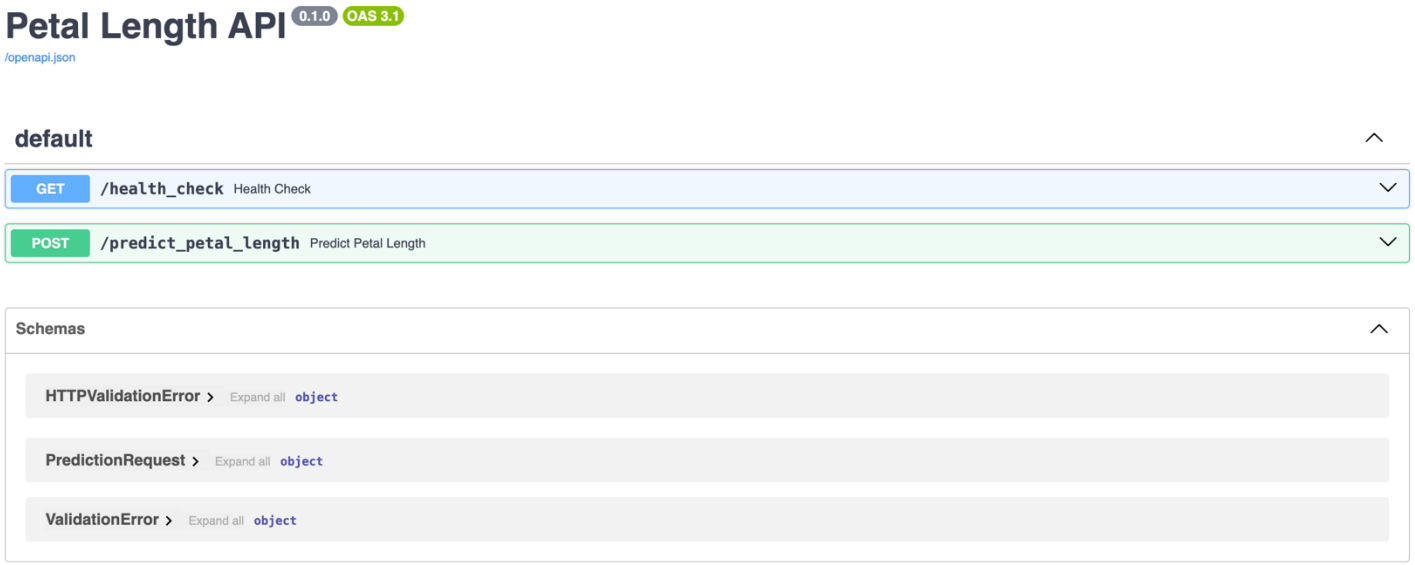
FastAPI generates docs automatically:

Swagger UI (/docs):
Interactive testing
Try endpoints in browser
See schemas

ReDoc (/redoc):
Clean reference docs
Professional appearance

Generated from:
Route definitions
Pydantic models
Type hints
Docstrings

Always up to date



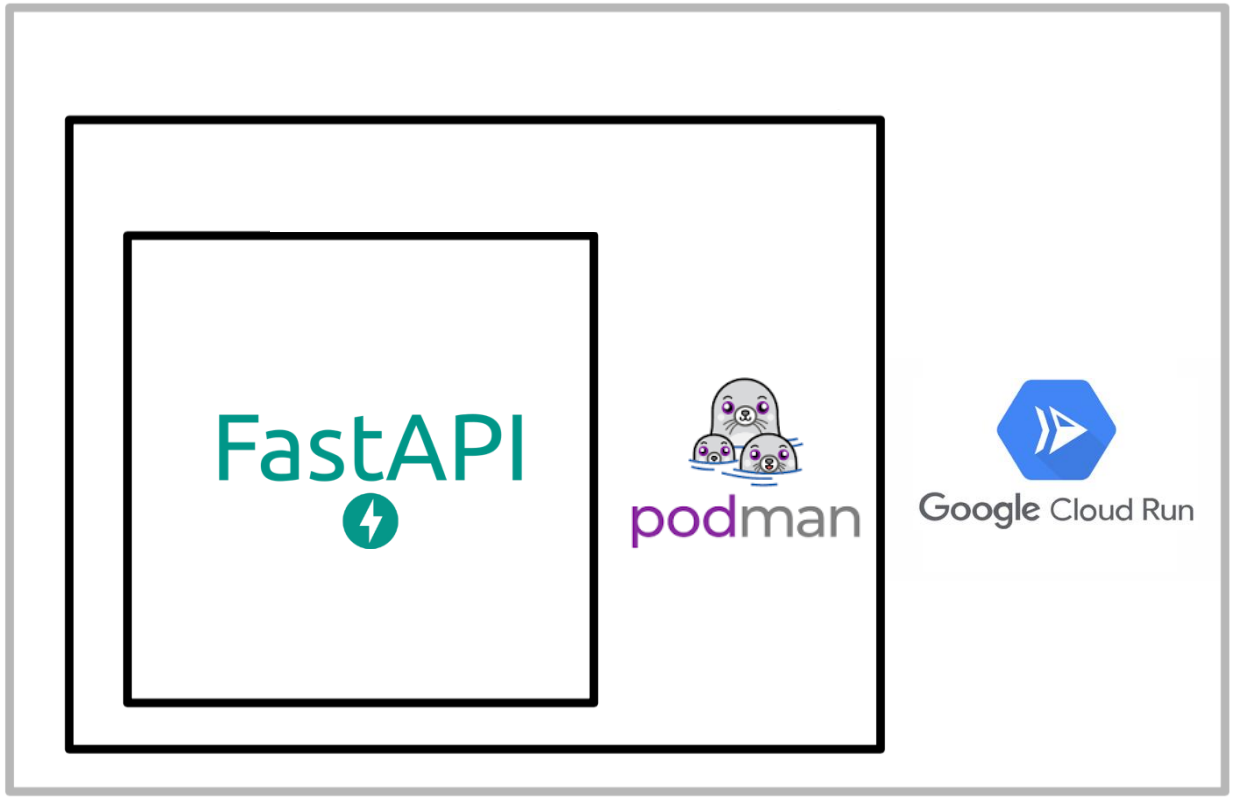
Batch Predictions

```
class BatchRequest(BaseModel):
    instances: list[list[float]]

@app.post("/predict/batch")
def predict_batch(req: BatchRequest):
    preds = model.predict(req.instances)
    return {
        "predictions": [float(p) for p in preds],
        "count": len(preds)
    }
```

More efficient
Reduce network overhead
Bulk processing
Add batch size limits

How we are going to deploy



Cloud Run Requirements

1. Listen on port from $PORT env var:

```
port = int(os.getenv("PORT", 8080))
uvicorn.run(app, host="0.0.0.0", port=port)
```

2. Be stateless:
No local file storage
Use Cloud Storage for files
Use databases for state

3. Start quickly:
Load models at startup
Optimize initialization
Use health checks

4. Handle graceful shutdown:
Finish requests in progress
Clean up resources

Dockerfile for Cloud Run

```
FROM python:3.11-slim

WORKDIR /app

COPY pyproject.toml README.md ./
COPY app ./app
COPY models ./models

RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir .

EXPOSE 8080

CMD ["python", "-m", "app.main"]
```

Building with Podman

Build the image:

```
podman build -t fastapi-418:latest .
```

Build for AMD64/x86_64:

```
podman build --platform linux/amd64 -t natelangholz/fastapi-418:latest .
```

Run the container locally:

```
podman run --rm -p 8080:8080 fastapi-418:latest
```

Use compose:

```
podman compose up --build
```

Deploying to Cloud Run

Essentially, once we have built the image and pushed to our registry we can build in the same fashion we did with our streamlit app.

Create a service

Link to image container url

Allow public requests

Confirm your ports are matching

Create deployment

Environment Variables

Set via gcloud:

```
gcloud run services update myapi \
  --set-env-vars API_KEY=secret123,DEBUG=false
```

Or in deployment:

```
--set-env-vars KEY1=value1,KEY2=value2
```

For secrets, use Secret Manager:

```
gcloud run services update myapi \
  --set-secrets API_KEY=api-key:latest
```

Never hardcode secrets
Use Secret Manager for sensitive data
Environment vars for config
Different values per environment

Custom Domains

Add custom domain:

1. Verify domain ownership:
gcloud domains verify example.com

2. Map domain to service:
```
gcloud run domain-mappings create \
  --service myapi \
  --domain api.example.com \
  --region us-central1
```

3. Update DNS:
Add CNAME record pointing to Cloud Run

4. SSL certificate:
Automatically provisioned
Auto-renewal

Professional appearance
HTTPS included
Easy setup

Scaling Configuration

Auto-scaling settings:

```
--min-instances 1  # Always warm
--max-instances 100  # Scale limit
--concurrency 80  # Requests per instance
--cpu 2  # CPU allocation
--memory 4Gi  # Memory limit
--timeout 300s  # Request timeout
```

Scaling strategies:
Min instances = 0: Cost effective
Min instances > 0: No cold starts
Max instances: Cost control
Concurrency: Throughput tuning

Monitor and adjust
Balance cost and performance

Monitoring Cloud Run

Built-in monitoring:

Cloud Console:
Request count
Latency
Error rate
Instance count
CPU/memory usage

Cloud Logging:
Application logs
Request logs
Error logs

Cloud Trace:
Request tracing
Latency analysis

Set up alerts:
High error rate
High latency
Cost thresholds

Monitor continuously
Set up alerts
Review regularly

API Best Practices

Authentication

API Key:

```
from fastapi import Header

@app.get("/predict")
def predict(api_key: str = Header(...)):
    if api_key != os.getenv("API_KEY"):
        raise HTTPException(401)
    # Your code
```

API keys: Simple, service-to-service

JWT:

```
from fastapi.security import HTTPBearer

security = HTTPBearer()

@app.get("/predict")
def predict(creds = Depends(security)):
    token = creds.credentials
    payload = jwt.decode(token, SECRET)
    # Your code
```

JWT: Secure, user authentication

Rate Limiting

```
from slowapi import Limiter

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter

@app.get("/predict")
@limiter.limit("100/minute")
def predict(request: Request):
    # Your code
    pass
```

Why rate limit:
Prevent abuse
Protect resources
Fair usage
Prevent DDoS

Different limits:
Expensive ops: Lower
Cheap ops: Higher
Authenticated: Higher
Public: Lower

CORS Configuration

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "https://myapp.com",
        "https://staging.myapp.com"
    ],
    allow_credentials=True,
    allow_methods=["GET", "POST"],
    allow_headers=["*"],
)

Development: allow_origins=["*"]
Production: Specific origins only
```

CORS enables browser requests
Security feature
Configure properly

Error Handling

```
from fastapi import HTTPException

@app.post("/predict")
def predict(req: PredictionRequest):
    try:
        if model is None:
            raise HTTPException(503, "Model not loaded")

        pred = model.predict([req.features])
        return {"prediction": float(pred[0])}

    except ValueError as e:
        raise HTTPException(400, f"Invalid input: {e}")
    except Exception as e:
        raise HTTPException(500, "Internal error")
```

Use correct status codes:
200: Success
400: Client error
401: Unauthorized
500: Server error
503: Unavailable

Input Validation

```
from pydantic import validator, Field

class PredictionRequest(BaseModel):
    features: list[float] = Field(min_items=10, max_items=10)

    @validator("features")
    def validate(cls, v):
        if any(not math.isfinite(x) for x in v):
            raise ValueError("Must be finite")
        if any(x < 0 or x > 100 for x in v):
            raise ValueError("Range 0-100")
        return v
```

Validate everything:
Data types
Value ranges
String patterns
List lengths

Never trust input
Validate before processing

Logging

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logger = logging.getLogger(__name__)

@app.post("/predict")
def predict(req: PredictionRequest):
    logger.info(f"Prediction request: {len(req.features)} features")

    try:
        pred = model.predict([req.features])
        logger.info("Prediction completed")
        return {"prediction": float(pred[0])}
    except Exception as e:
        logger.error(f"Prediction failed: {e}")
        raise
```

Log:
Requests/responses
Errors
Performance
Authentication

Health Checks

```
@app.get("/health")
def health():
    return {"status": "healthy"}

@app.get("/ready")
def ready():
    try:
        if model is None:
            raise HTTPException(503, "Model not loaded")

        # Test prediction
        test = [0.0] * 10
        _ = model.predict([test])

        return {"status": "ready"}
    except Exception as e:
        raise HTTPException(503, str(e))
```

Health: Is process running?
Ready: Can handle traffic?

Used by:
Load balancers
Orchestrators
Monitoring

Deployment Checklist

Before deploying:

- Authentication implemented
- Rate limiting configured
- CORS set up properly
- Error handling complete
- Logging in place
- Health checks working
- Tests passing
- Documentation complete
- Environment variables set
- Secrets in Secret Manager
- Resource limits configured
- Monitoring set up
- Alerts configured

Don't skip steps
Production is different
Be prepared

Common Pitfalls

Avoid these mistakes:

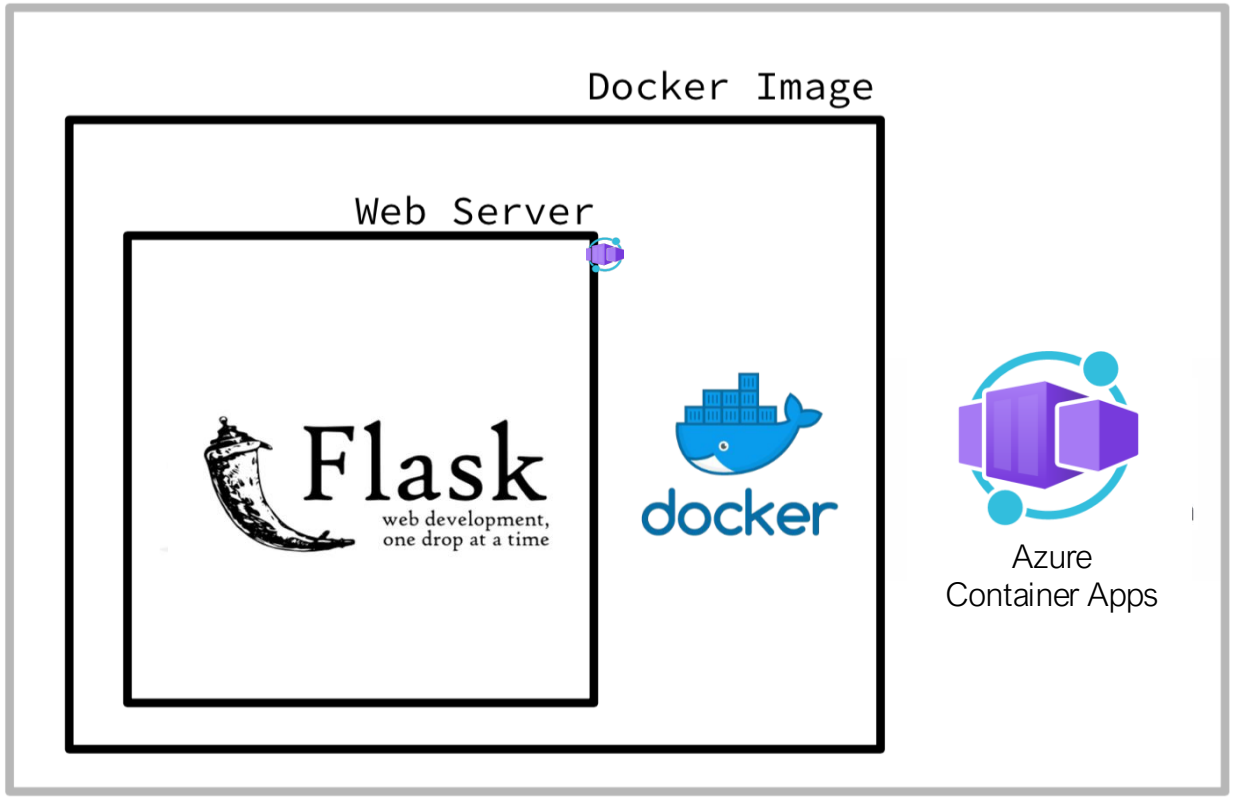
1. No authentication
2. Hardcoded secrets
3. No rate limiting
4. Poor error handling
5. No logging
6. No health checks
7. No tests
8. Wrong CORS config
9. No monitoring
10. Loading model per request
11. No input validation
12. Blocking async endpoints
13. Large container images
14. Running as root
15. No resource limits

Best Practices

Security:
Authenticate all endpoints
Validate all inputs
Use secrets manager
Run as non-root
Scan for vulnerabilities

Performance:
Load models at startup
Use caching
Async for I/O
Batch predictions
Optimize containers

Reliability:
Health checks
Error handling
Logging
Monitoring
Graceful shutdown

Maintainability:
Clear structure
Good documentation
Tests
Versioning

These are all interchangeable with slightly different configurations. We could also deploy a model endpoint as such.